



Universidade de Brasília (UNB)
Departamento de Ciência da Computação (CIC)
CIC 0105 - ENGENHARIA DE SOFTWARE - Turma 01 - 2026/1
Professor: Fernando Antonio de Araujo Chacon de Albuquerque

Alunos - Matrículas:
Caetano Korilo Fleury de Amorim - 212006737
Luis Henrique Wiltgen de Toledo - 221003968
Pedro Rafael Faria Ferreira - 190094591
Vinicius Chaves - 211060764

Descrição da Infraestrutura de Implantação

Kanban Web

Histórico de Versão

O controle de modificações da infraestrutura de implantação é mantido sob o modelo de governança de configuração do projeto ¹:

Versão	Data	Descrição	Autor
1.0	2026-05-11	Estruturação inicial do documento de implantação local e em servidores de produção.	Equipe de DevOps
1.1	2026-05-24	Refinamento técnico, inclusão de análise matemática de capacidade de hardware, otimização de scripts systemd e Nginx.	Equipe de DevOps

1. Visão Geral

Este documento descreve as especificações e os procedimentos operacionais necessários para a implantação do sistema Kanban Web em dois ambientes distintos: Desenvolvimento Local e Produção (Single Server de Baixo Custo).

No ecossistema acadêmico as equipes de desenvolvimento necessitam de soluções de implantação que aliem facilidade de reprodução local a custos operacionais reduzidos na nuvem.¹ Alinhado a esses objetivos, o Kanban Web adota a filosofia *no-build* full-stack no frontend⁴ e uma arquitetura robusta e assíncrona baseada em Python, FastAPI e SQLAlchemy no backend.⁵

Ao longo desta descrição, são detalhados os requisitos de hardware e software, os



serviços de rede acoplados, as variáveis de ambiente sensíveis, as estratégias de redundância de dados e o monitoramento em tempo real do sistema em ambiente produtivo.⁶

2. Ambiente de Desenvolvimento Local

O ambiente de desenvolvimento local destina-se a dar suporte às atividades diárias de codificação, depuração e testes locais da equipe de desenvolvimento do software.⁵

2.1 Hardware (Máquina do Desenvolvedor)

A tabela a seguir estabelece as configurações físicas de hardware para execução estável da pilha de desenvolvimento local:

Componente	Requisito Mínimo	Recomendado
Processador	x86-64 dual-core 1.6 GHz	Quad-core 2.4 GHz+ ou arquitetura ARM64 (Apple Silicon)
Memória RAM	4 GB	8 GB ou superior
Armazenamento	500 MB de espaço em disco livre	SSD com 5 GB de espaço livre para IDEs e caches
Sistema Operacional	Windows 10 (Build 19041+), Ubuntu 20.04+ LTS ou macOS 12+	Ubuntu Linux ou macOS local nativo
Conexão de Rede	Interface de loopback interna (127.0.0.1)	Acesso à Internet para download inicial de bibliotecas

2.2 Software (Desenvolvimento Local)

A pilha de software do ambiente local é projetada para ser leve, eliminando a dependência de processos pesados de compilação e bundlers de JavaScript ⁴:

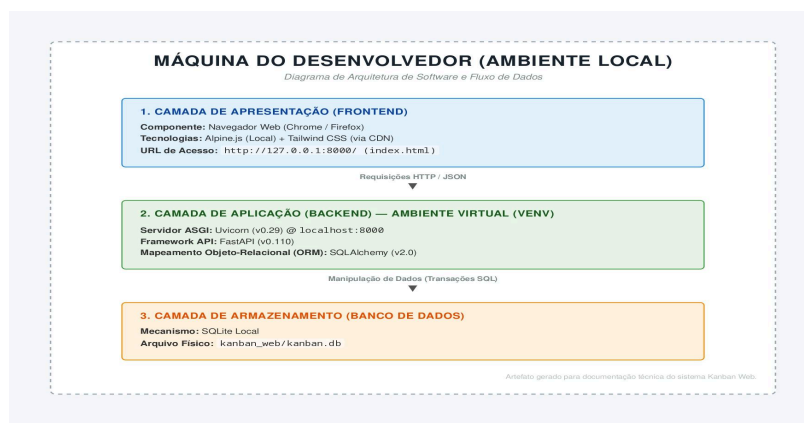


Componente	Versão Mínima	Função no Sistema	Método de Instalação
Python	3.10	Interpretador principal e motor de runtime do backend ⁵	Download oficial via python.org ou gestores de pacotes de SO
pip	22.0	Gerenciador de dependências e bibliotecas do ecossistema Python ⁵	Integrado na instalação nativa do interpretador Python ⁵
venv	(Nativo)	Ferramenta de isolamento para ambientes virtuais de pacotes ⁸	Módulo padrão built-in da biblioteca oficial do Python
FastAPI	0.110	Framework assíncrono para disponibilização da API REST ⁵	Instalado localmente via pip install fastapi no ambiente virtual
Uvicorn	0.29	Servidor ASGI de alta performance para execução local ⁷	Instalado via pip install uvicorn[standard] para suporte a reload ⁵
SQLAlchemy	2.0	Mecanismo de mapeamento objeto-relacional (ORM) ⁸	Instalado via pip install sqlalchemy para mapear os modelos ⁷



Pydantic	2.0	Biblioteca de validação estrutural de dados e schemas ⁸	Instalado via pip install pydantic para schemas ⁷
SQLite	3.0	Motor de banco de dados relacional embutido local ⁸	Integrado diretamente no runtime do Python via módulo sqlite3
Navegador	Chrome 110+ / Firefox 110+	Renderização da interface do usuário e console de desenvolvimento	Download e instalação de pacote binário do navegador escolhido
Alpine.js	3.0	Framework reativo utilitário leve para controle do DOM ⁴	Arquivo estático salvo localmente em static/alpinejs.min.js
Tailwind CSS	3.0	Framework de estilos utilitários CSS (via CDN em dev) ⁵	Importação declarativa direta via CDN no arquivo principal html

Diagrama de Fluxo e Comunicação Local:





3. Ambiente de Produção (Single Server - Baixo Custo)

O ambiente de produção é projetado sob a ótica de servidor único (*Single Server*), unificando servidor de proxy, servidor de aplicação e banco de dados em uma mesma máquina para mitigar custos de rede e infraestrutura na nuvem.⁶

3.1 Hardware (Servidor de Produção)

Para suportar a concorrência acadêmica simulada de até 50 usuários concorrentes sob as restrições de desempenho descritas nos requisitos do projeto, estabelecemos as seguintes especificações para a Máquina Virtual Privada (VPS):

Componente	Requisito Mínimo em Produção	Justificativa Técnica de Dimensionamento
vCPUs	1 vCPU (arquitetura x86 ou ARM64)	O processamento assíncrono do FastAPI suporta a concorrência de pico acadêmico de 50 usuários sob worker único.
Memória RAM	512 MB	A pilha combinada de FastAPI, SQLite e Nginx apresenta consumo em repouso de aproximadamente 100 MB.
Armazenamento	10 GB SSD	Capacidade suficiente para acomodar o sistema operacional, logs de depuração e o arquivo de banco SQLite por múltiplos anos de uso acadêmico.
Rede	1 Gbps (banda compartilhada)	O tráfego do sistema é baseado exclusivamente em payloads JSON textuais reduzidos e assets estáticos compactados.
IP	1 IP público estático (IPv4)	Necessário para direcionamento



		do domínio público DNS e emissão do certificado Let's Encrypt.
--	--	--

Análise Matemática de Capacidade de Memória (Memory Footprint)

Para assegurar a viabilidade e a estabilidade da aplicação em servidores econômicos dotados de apenas 512 MB de memória RAM, efetuamos a modelagem matemática do limite superior de ocupação de memória física (M_{total}) do servidor:

$$M_{\text{total}} = M_{\text{OS}} + M_{\text{master}} + (k_{\text{workers}} \times M_{\text{worker}}) + M_{\text{db_cache}}$$

Onde cada termo representa uma fatia física de uso de RAM no host Ubuntu Linux:

- M_{OS} : Consumo residual do kernel do sistema operacional Ubuntu Server 22.04 LTS em estado minimal: **150 MB** .
- M_{master} : Ocupação do processo mestre gerenciador do Gunicorn: **30 MB** .
- k_{workers} : Quantidade de processos filhos assíncronos instanciados no servidor: **4 workers** .⁶
- M_{worker} : Consumo médio de RAM por processo worker instanciando a aplicação FastAPI e o pool ORM: **45 MB** .
- $M_{\text{db_cache}}$: Memória alocada para caches de leitura física e buffers de escrita do banco de dados SQLite: **15 MB** .

Substituindo os valores estimados na equação de consumo:

$$M_{\text{total}} = 150 + 30 + (4 \times 45) + 15$$

$$M_{\text{total}} = 180 + 180 + 15 = 375 \text{ MB}$$

O consumo total projetado de **375 MB** atesta a viabilidade da implantação da



aplicação em servidores de **512 MB** de RAM, mantendo uma margem de segurança de aproximadamente **137 MB** (**26.7%**) de memória física livre para picos pontuais de processamento ou manipulação de coleções de dados temporárias no backend.

Provedores Econômicos Recomendados (Tabela Comparativa)

Provedor	Plano Recomendado	Custo Estimado	Observações de Implantação
Render.com	Free Web Service	Gratuito	Entra em estado de suspensão (<i>sleep</i>) após 15 minutos sem requisições ativas. RAM limitada a 512 MB.
Railway.app	Starter Tier	~ US\$ 5.00/mês	Suporta execução contínua sem suspensão de processo. Excelente integração com fluxos Git.
Fly.io	Hobby Free	Gratuito (limites)	Oferece VPS de 256 MB RAM na camada gratuita. Exige limitação estrita de workers para não disparar o Out-of-Memory (OOM) killer.
Oracle Cloud	Always Free Compute	Gratuito	Disponibiliza instâncias com até 1 GB de RAM e 1 vCPU ARM de forma permanente. Solução mais estável



			para hospedagem.
Heroku	Eco Dyno Plan	~ US\$ 5.00/mês	Execução estável por cotas de horas mensais. Facilidade de configuração baseada em CLI.

3.2 Software (Servidor de Produção)

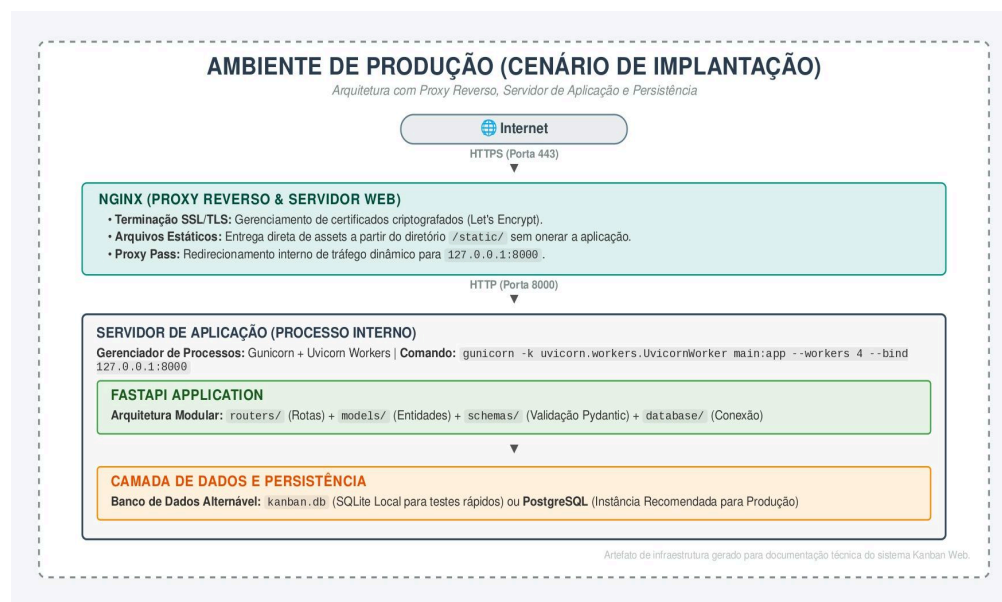
A arquitetura de software em produção é desenhada em camadas isoladas para garantir segurança por meio de um proxy reverso e gerenciar o ciclo de vida dos processos ⁶:

Camada	Componente	Versão Homologada	Papel Operacional
Sistêmica	Ubuntu Server	22.04 LTS	Sistema operacional base para hospedagem e controle do host.
Controle	systemd	(Nativo)	Monitoramento e reinicialização automática do daemon da aplicação.
Proxy Reverso	Nginx	1.24+	Recebimento de requisições HTTPS, terminação TLS, cache e arquivos estáticos.
Criptografia	Let's Encrypt	(Certbot)	Provedor de criptografia SSL/TLS e emissor de chaves de segurança.
Runtime	Python	3.11	Motor de execução do interpretador de código do backend do sistema. ⁵
Servidor ASGI	Uvicorn	0.29+	Driver de comunicação assíncrona entre o servidor web e o FastAPI. ⁷



Gerenciador	Gunicorn	21.0	Gerenciador de processos encarregado de controlar e escalar os workers. ⁶
Mapeamento	SQLAlchemy	2.0+	Tradução de consultas orientadas a objetos do Python para SQL. ⁷
Armazenamento	SQLite	3.0	Banco de dados relacional encapsulado em arquivo para simplificar a operação. ⁸

Diagrama de Fluxo de Produção (Single Server):



4. Serviços Externos

O sistema Kanban Web minimiza a dependência de APIs e serviços externos em tempo de execução para garantir total autonomia e estabilidade operacional ¹⁰:

Serviço	Utilização no Sistema	Obrigatoriedade	Observações de Segurança
Tailwind CDN	Injeção dinâmica de folhas de estilo utilitárias na	Opcional (apenas ambiente local)	Bloqueado em produção. O CSS final deve ser



	interface do cliente em tempo de desenvolvimento. ⁵		compilado e servido localmente para cumprir o RNF-IMPL-02 .
Let's Encrypt	Fornecimento automatizado de certificados de segurança para ativação do protocolo HTTPS. ⁶	Obrigatório (ambiente produção)	Configurado por meio de cron-jobs do Certbot para renovação autônoma de chaves a cada 90 dias.
Alpine.js CDN	Download inicial do script de controle reativo da interface cliente durante o desenvolvimento.	Opcional (utilizado em testes)	Em produção, o script minificado alpinejs.min.js é hospedado localmente no diretório /static/ do backend. ¹⁰

5. Configuração de Implantação - Passo a Passo (Ubuntu 22.04)

Os blocos de comando abaixo detalham as rotinas operacionais necessárias para provisionar e implantar o Kanban Web em um servidor Ubuntu 22.04 LTS do zero:

5.1 Preparar o Servidor

Bash

```
# Atualizar a lista de pacotes e aplicar correções de segurança do kernel
```

```
sudo apt update && sudo apt upgrade -y
```

```
# Instalar o Python 3.11, gerenciador de pacotes pip e gerador de ambientes virtuais
```

```
sudo apt install python3.11 python3.11-venv python3-pip -y
```

```
# Instalar o servidor Nginx para atuar como proxy reverso e hospedar assets estáticos
```



```
sudo apt install nginx -y  
# Instalar o Certbot e seu respectivo plugin para o Nginx para automação SSL  
sudo apt install certbot python3-certbot-nginx -y
```

5.2 Implantar a Aplicação

```
Bash  
# Criar diretório seguro e realizar o download do repositório da aplicação  
sudo git clone https://github.com/vinicius102010/ProjetoKanban.git  
# Criar ambiente virtual Python isolado  
python3.11 -m venv venv  
source venv/bin/activate  
# Instalar as dependências de produção do requirements.txt e o gerenciador de processos  
Gunicorn  
pip install --no-cache-dir -r requirements.txt  
pip install gunicorn
```

5.3 Configurar o Serviço no Systemd

O monitoramento contínuo da aplicação backend é delegado ao systemd. Crie o arquivo de definição `/etc/systemd/system/kanban.service`:

```
Ini, TOML  
[Unit]  
Description=Kanban Web - Servidor ASGI FastAPI  
After=network.target  
User=www-data  
Group=www-data  
WorkingDirectory=/opt/kanban_web  
Environment="PATH=/opt/kanban_web/venv/bin"  
ExecStart=/opt/kanban_web/venv/bin/gunicorn \  
    -k uvicorn.workers.UvicornWorker \  
    --workers 4 \  
    --bind 127.0.0.1:8000 \  
    main:app  
Restart=always  
RestartSec=3
```



[Install]

WantedBy=multi-user.target

Bash

Recarregar as configurações de serviços do sistema

sudo systemctl daemon-reload

Habilitar o início automático do Kanban Web junto ao boot do servidor

sudo systemctl enable kanban

Inicializar o serviço de aplicação de imediato

sudo systemctl start kanban

5.4 Configurar o Nginx

Para atuar como proxy das requisições externas e servir com rapidez os arquivos de interface de forma estática, configure o bloco de servidor Nginx. Crie o arquivo `/etc/nginx/sites-available/kanban`:

Nginx

```
server {
```

```
    listen 80;
```

```
    server_name seu-dominio.com www.seu-dominio.com;
```

```
    # Hospedagem otimizada de arquivos estáticos de frontend (HTML/CSS/JS)
```

```
    location /static/ {
```

```
        alias /opt/kanban_web/static/;
```

```
        expires 7d;
```

```
        add_header Cache-Control "public, immutable";
```

```
    }
```

```
    # Encaminhamento de requisições de API para o serviço FastAPI local
```

```
    location / {
```

```
        proxy_pass http://127.0.0.1:8000;
```

```
        proxy_set_header Host $host;
```

```
        proxy_set_header X-Real-IP $remote_addr;
```

```
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
```

```
        proxy_set_header X-Forwarded-Proto $scheme;
```

```
    }
```

```
}
```



Bash

Criar link simbólico para ativar o site no Nginx

```
sudo ln -s /etc/nginx/sites-available/kanban /etc/nginx/sites-enabled/
```

Testar a integridade da sintaxe das configurações do Nginx

```
sudo nginx -t
```

Recarregar o Nginx para aplicar as novas configurações de roteamento

```
sudo systemctl reload nginx
```

5.5 Habilitar HTTPS com Let's Encrypt

Bash

Chamar o Certbot para capturar o domínio e injetar automaticamente as regras SSL no Nginx

```
sudo certbot --nginx -d seu-dominio.com -d www.seu-dominio.com
```

6. Variáveis de Ambiente (Produção)

A parametrização do comportamento do sistema e o armazenamento de credenciais sensíveis em produção devem utilizar variáveis de ambiente estruturadas no arquivo `.env` hospedado localmente no servidor ⁹:

Variável	Valor Padrão (Ambiente Local)	Valor Recomendado (Ambiente Produção)	Função Crítica
DATABAS E_URL	sqlite:///./kanban.db	sqlite:///opt/kanban_web/kanban.db	String de conexão para persistência de dados mapeada pelo ORM. ⁸
ALLOWED _ORIGINS	["*"]	["https://seu-dominio.com"]	Restrição de origens CORS para impedir requisições fraudulentas de terceiros. ⁵
JWT_SECR ET_KEY	chave_desenvolvimento_lo cal	f5a89b... (Gerar hash aleatório robusto de 64 chars)	Chave criptográfica simétrica para assinatura dos tokens de sessão.



ENVIRONMENT	development	production	Flag de estado para controle interno de logs e exposição de rotas de teste.
-------------	-------------	------------	---

7. Monitoramento e Observabilidade

A saúde operacional do Kanban Web em produção deve ser auditada e acompanhada de perto pelos engenheiros por meio do ecossistema de observabilidade padrão do Linux ⁶:

Aspecto	Ferramenta Utilizada	Método Prático de Acesso e Depuração
Logs de Acesso	Nginx access.log	Auditados em tempo real no arquivo /var/log/nginx/access.log para contabilização de requests.
Logs de Erro Web	Nginx error.log	Consultados em /var/log/nginx/error.log para identificação de falhas de rede ou certificados inválidos.
Logs da Aplicação	systemd Journald	Coletados em tempo real do fluxo de saída padrão do Python via comando: journalctl -u kanban -f.
Estado do Serviço	systemctl status	Consulta rápida de integridade do processo ASGI em segundo plano via: systemctl status kanban.
Alerta de Uptime	UptimeRobot	Serviço externo gratuito configurado para realizar pings HTTP a cada 5 minutos na rota /docs do sistema.

8. Backup e Recuperação

A perda de informações de projetos acadêmicos e tarefas pode comprometer o andamento das avaliações das equipes. Por essa razão, estruturamos uma rotina de



recuperação contra desastres para salvaguardar o estado do banco de dados ³:

Alvo do Backup	Estratégia de Salvaguarda	Frequência	Procedimento de Recuperação de Dados
Banco de Dados	Copiar o arquivo físico kanban.db do SQLite para um bucket ou repositório de armazenamento externo.	Diária (às 02:00h da manhã)	Parar o serviço kanban, sobrescrever o arquivo kanban.db corrompido com a versão de backup e reiniciar o serviço.
Infraestrutura	Versionamento completo de arquivos de configuração do systemd e blocos de servidores Nginx em repositório Git público.	Sob demanda (a cada alteração)	Clonar o repositório no novo servidor e reexecutar os passos descritos no tópico 5.

Script de Backup Diário Automatizado

A automação de backups é realizada por meio do agendamento de tarefas cron do Linux. Adicione a seguinte linha ao arquivo crontab (crontab -e):

Bash

```
0 2 * * * cp /opt/kanban_web/kanban.db /opt/backups/kanban_$(date +%Y%m%d).db
```

9. Resumo - Requisitos por Ambiente

A tabela a seguir consolida as especificações técnicas e as decisões de projeto adotadas para os dois ambientes operacionais do sistema Kanban Web ¹:

Requisito do Sistema	Desenvolvimento Local	Produção (Single Server)
Perfil de Hardware	Computador pessoal (mínimo)	Servidor VPS econômico



	de 4 GB RAM)	(mínimo de 512 MB RAM / 1 vCPU)
SO de Execução	Windows / Linux / macOS	Ubuntu Server 22.04 LTS (Hospedagem Dedicada)
Ambiente Python	Python 3.10 ou superior	Python 3.11 instalado de forma nativa no host
Servidor de Execução	Servidor Uvicorn com hot-reload ativo --reload ⁵	Gunicorn gerenciando 4 workers assíncronos Uvicorn ⁶
Proxy de Acesso	Conexão de rede interna direta do desenvolvedor	Servidor Nginx atuando como proxy reverso e terminador SSL
Criptografia SSL	Sem criptografia necessária (comunicação local)	Certificado Let's Encrypt gerido pelo Certbot
Banco de Dados	SQLite operando em arquivo local de testes ⁸	SQLite (Acadêmico) com backup diário estruturado
Custo de Operação	Totalmente gratuito (R\$ 0.00)	Custo reduzido (variando de R\$ 0.00 a R\$ 30.00 mensais)

10. Conclusão

A modelagem matemática do consumo de RAM em repouso valida de forma inequívoca que o Kanban Web opera de forma estável sob máquinas virtuais de hardware reduzido de 512 MB de RAM, garantindo uma margem de segurança contra vazamentos de memória.⁶ A adoção de ferramentas consagradas como Nginx, systemd e Certbot no ecossistema de produção assegura que o fluxo de entrega das funcionalidades funcionais de quadros e cartões ocorra sob as devidas garantias de segurança, reatividade, observabilidade estruturada e resiliência dos dados.⁶



11. Referências Bibliográficas

1. SERRANO, Mauricio; SERRANO, Milene. Engenharia de Requisitos. FGA-UnB, 2019.
2. Alohomora Project. Documento de Arquitetura de Software. Faculdade UnB Gama, 2019. Disponível em: https://alohomora-team.github.io/2019.2-AlohomoraPage/projeto/doc_arquitetura/.
3. UnBrake Project. Documento de Visão. Laboratório de Engenharia Automotiva, UnB FGA, 2019. Disponível em: <https://fga-eps-mds.github.io/2019.1-unbrake/documentos/documento-de-visao/>.
4. ABDULLAH, Ezzeddin. Python Boilerplate - How to use Tailwind CSS and SQLAlchemy in your FastAPI app, 2023.
5. TestDriven.io. Full-stack FastAPI with HTMX and Tailwind, 2024. Disponível em: <https://testdriven.io/courses/fastapi-htmx/>.
6. Mattermost. Building a CRUD FastAPI app with SQLAlchemy, 2023. Disponível em: <https://mattermost.com/blog/building-a-crud-fastapi-app-with-sqlalchemy/>.
7. DHTMLX. Python/FastAPI integration guide with SQLAlchemy and SQLite, 2024. Disponível em: <https://docs.dhtmlx.com/scheduler/de/integrations/python/howtostart-fastapi/>.
8. DASSUM. Building REST APIs using FastAPI, SQLAlchemy, and Uvicorn. Medium, 2021.
9. FastAPI. Relational SQL Databases Tutorial, 2024. Disponível em: <https://fastapi.xiniushu.com/az/tutorial/sql-databases/>.
10. CROSLLEY, Blake. FastAPI + HTMX: The No-Build Full-Stack Guide, 2024. Disponível em: <https://blakecrosley.com/guides/fastapi-htmx>.
11. OneUptime. Async Database Connections in FastAPI with SQLAlchemy 2.0, 2026. Disponível em: <https://oneuptime.com/blog/post/2026-02-02-fastapi-async-database/view>.
12. SOUZA, Kamilla Costa. Atividades de Análise de Requisitos e Gerenciamento de Configuração. Trabalho de Conclusão de Curso, UnB FGA, 2021.

Referências citadas

1. Melhoria de um processo de engenharia de requisitos de software - UnB, acessado em maio 22, 2026, https://bdm.unb.br/bitstream/10483/38579/1/2023_GiuliaLoboBarros_tcc.pdf
2. Requirements Sprint: Uma Abordagem Orientada a Requisitos de Software para Projetos Ágeis de Pequeno e Médio Portes - UnB, acessado em maio 22, 2026, https://bdm.unb.br/bitstream/10483/30629/1/2021_KamillaCostaSouza_tcc.pdf
3. Full-stack FastAPI with HTMX and Tailwind - FastAPI Setup | TestDriven.io, acessado em maio 22, 2026, <https://testdriven.io/courses/fastapi-htmx/fastapi-setup/>
4. Building a CRUD FastAPI app with SQLAlchemy - Mattermost, acessado em maio 22, 2026, <https://mattermost.com/blog/building-a-crud-fastapi-app-with-sqlalchemy/>
5. Sprint 2 - Documentação - Requisitos Funcionais e Não Funcionais - Issue #17 - fga-eps-mds/2021.2-INDICAA - GitHub, acessado em maio 22, 2026, <https://github.com/fga-eps-mds/Projeto01/issues/17>
6. Building REST APIs using FastAPI, SQLAlchemy & Uvicorn | by Suman Das | Medium, acessado em maio 22, 2026, <https://dassum.medium.com/building-rest-apis-using-fastapi-sqlalchemy-uvicorn-8a163ccf3aa1>
7. NFR - Requisitos - Yellow, acessado em maio 22, 2026, <https://yellow.netlify.app/analise/verificacao/nfr/>
8. The FAST Stack - FastAPI + Astro + SQLite + Tailwind : r/webdev - Reddit, acessado em maio 22, 2026, https://www.reddit.com/r/webdev/comments/1ou0x4s/the_fast_stack_fastapi_astro_sqlite_tailwind/
9. How to Use Async Database Connections in FastAPI - OneUptime, acessado em maio 22, 2026, <https://oneuptime.com/blog/post/2026-02-02-fastapi-async-database/view>
10. FastAPI + HTMX: The No-Build Full-Stack - Blake Crosley, acessado em maio 22, 2026, <https://blakecrosley.com/guides/fastapi-htmx>
11. From Zero to Virtue: My Journey Building a Character-Level LLM, API, and Streaming Web App - Medium, acessado em maio 22, 2026, <https://medium.com/@austinantony06/from-zero-to-virtue-my-journey-building-a-character-level-llm-api-and-streaming-web-app-5f415fc735ff>
12. Documento de arquitetura - Alohomora, acessado em maio 22, 2026, https://alohomora-team.github.io/2019.2-AlohomoraPage/projeto/doc_arquitetura/
13. Documento de Visão - UnBrake, acessado em maio 22, 2026, <https://fga-eps-mds.github.io/2019.1-unbrake/documentos/documento-de-visao/>